# n = 19 tau6 index calculus end to end in native code (N2 against the current row's arm N): frozen protocol

2026-10-21

## n = 19 tau6 index calculus end to end in native code: frozen protocol

**Status (2026-10-21): frozen and launched; no result yet.** The protocol (`protocol.json`) was frozen at 2026-10-21T01:12:27Z, before any timing. The workflow is `n19-native-loop.yml`. Dispatch was refused (GitHub MCP: 403 "Resource not accessible by integration"), so the owner-approved push trigger launched it. The trigger is limited to branch `claude/n19-native-loop` and to this experiment's non-report paths. The run URL is in Launch.

### Question

The current n = 19 row is arm N of `volcano-n19-native-tau6-v3-20261020` (PR #1005, run 39387473809). It is Tier 1, with cold CPU B / N = 141.02 [136.75, 144.95] and cold `S` = 31.42 in that run's rho unit. N is Sage's `tauDlp` running the v3 native per-shift solve through ctypes.

The question: does a candidate **N2** lower the end-to-end cold CPU of one complete, verified ECDLP against N? N2 runs the whole pipeline in native code, with no Sage process. Every target, shift order, shift system, relation, matrix, kernel and log must stay unchanged.

### Profile of the baseline

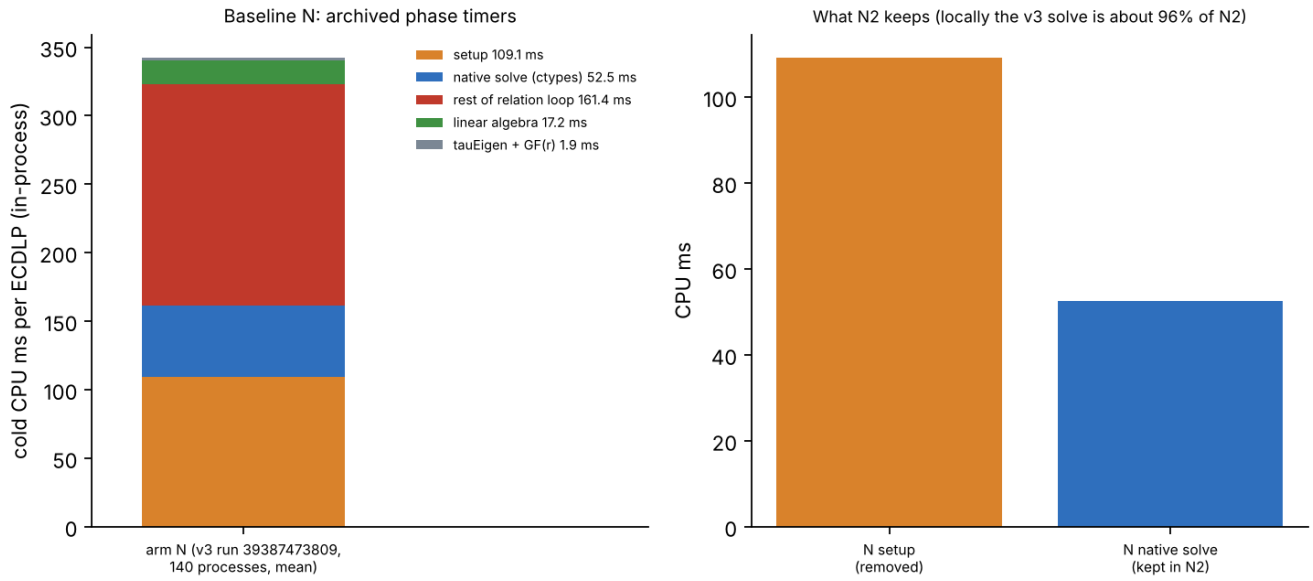



Figure 1: Where N's cold CPU goes. Left: the archived phase timers of all 140 N processes of run 39387473809. Right: what N2 keeps. Source: make_figures.py; vector: figures/profile.svg.

These come from the archived per-process timers of v3 run 39387473809 (140 N processes, means, in-process CPU). N's cold CPU is 342 ms:

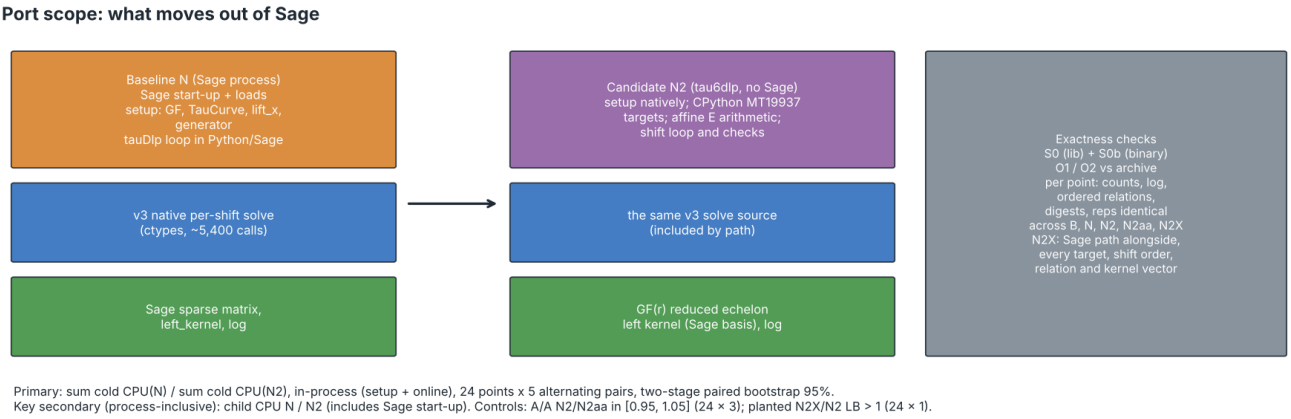
| part of N | ms | share | where it goes |
|---|---|---|---|
| setup | 109.1 | 31.9% | `GF(2^19)`, `TauCurve` (33 V' points: trace, half trace, 29 `lift_x`, then a Frobenius search per point), ctypes load, `generator` |
| native solve (ctypes) | 52.5 | 15.4% | about 5,446 calls at 9.6 $\mu$s each, Python call overhead included |
| rest of the relation loop | 161.4 | 47.2% | about 306 targets at about 527 $\mu$s each: two Sage scalar multiplications `al P + be Q`, `shuffle`, `fromBits` and base look-ups per solution, the Frobenius image and four sign combinations per candidate, and per-call Python overhead |
| linear algebra | 17.2 | 5.0% | Sage sparse matrix over GF(r), `left_kernel`, the log |
| `tauEigen` + `GF(r)` | 1.9 | 0.6% | before the loop |

The "rest of online" share of 52.7% in the v3 README is the last three rows of this table.

Outside the primary interval, each N process also pays Sage start-up and the source loads. In run 39387473809 the launcher's child CPU was 1.41 s per N process against a cold interval of 0.34 s.

A function-level profile of N on the runner is the untimed diagnostic `diagnosis/profile_n.sage`. It instruments the stages and checks that the instrumented run reproduces the plain run's counts and digests. It also runs cProfile, and `diagnosis/startup_probe.py` records Sage start-up per launch. It ran as diagnostic run 39522833864 (phase 1 of the workflow), and the smoke job runs it again on two points; neither is a gate. Its record goes under `diagnosis/run_github_*` when it is available.

## Port scope and exactness



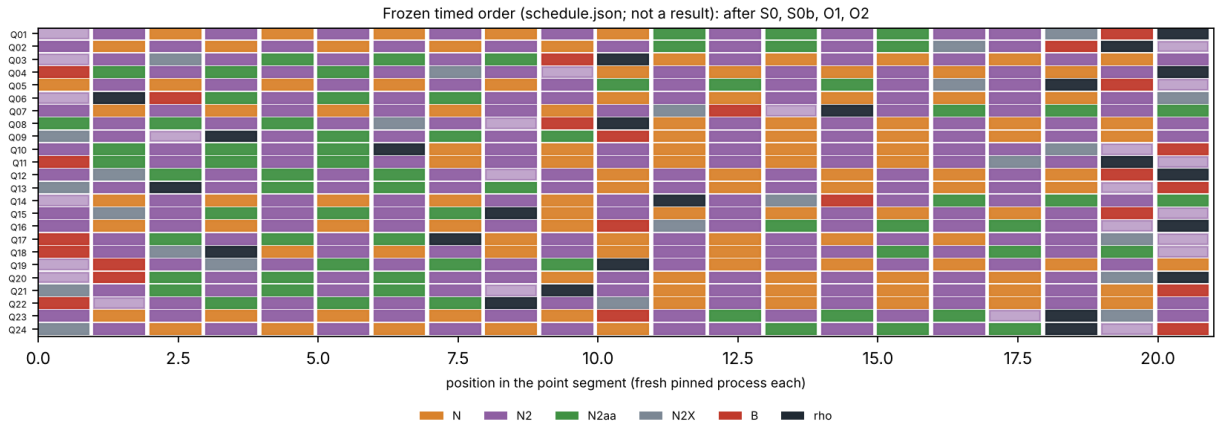




Figure 2: Port scope, checks and frozen run order (not a result). Source: make_figures.py; vector: figures/design.svg.

**The largest exact port.** Everything moves to native code (`native/`, crate `tau6-loop`): setup, the relation loop, the linear algebra and the log. N2 is one process of the `tau6dlp` binary, and Sage serves only as the oracle. The per-shift solve is **the v3 source file itself**: `native/src/lib.rs` includes `../../volcano-n19-native-tau6-v3-20261020/native/src/lib.rs` by path, together with its C ABI and its tests. Only the work around the solve changes.

| tau.sage step | N2 | why it is the same |
|---|---|---|
| V' base: x = c, $1 \le c < 64$, $\mathrm{Tr}(x + 1/x^2) = 0$, y = x·halfTrace | same arithmetic in GF(2^19) (4-bit comb plus table reduction, tested equal to v3 `gf_mul` on 20,000 products) | unique values |
| orbit representatives: smallest encoding; `E.lift_x(rx)` | `lift_x_all` returns y in ascending encoding | Sage's choice: fixed by the archived relations of all 29 columns (every sign agrees) and by the archived P |
| base table x → (P, column, e, ±1), first e | same search | unique |
| `generator(cur, 'E0')`: `Random('gen:E0')`, `lift_x(x, all=True)[0]`, cofactor 4 | CPython MT19937 port, same draws | equals the archived P (crate test) |
| `tauEigen`: root of $t^2 + t + 2$ with $\lambda G = \tau(G)$ | Tonelli–Shanks, then the check | unique root |

| tau.sage step | N2 | why it is the same |
|---|---|---|
| targets: `randrange(1, r)` twice, R = al P + be Q, skip zero, `shuffle(shifts)` in place | MT19937 + `_randbelow` + `shuffle` exactly as CPython 3.12 | compared with CPython for every protocol seed (test) |
| per shift: v3 variety in Sage's order, spurious counting, signs (1,1), (1,−1), (−1,1), (−1,−1), first hit | same loop over the v3 `Ctx::solve` | same solve source |
| rows column → Σ sign λ^e (insertion order), rhs (4al, 4be) | same | — |
| `M.left_kernel().basis()`, first vector with B ≠ 0, s = −A/B, s P == Q | reduced echelon basis of the left kernel over GF(r) | the echelon basis of a space is unique; the planted arm compares every vector with Sage's |

**Checks before freezing** (local VM, no Sage; not timing evidence):

- **Crate tests.** `cargo test --release`: 15 of 15 pass. These include the 7 v3 solve tests, MT19937 against the reference vector and CPython vectors, SHA-512 and SHA-256, the field and the group law, the archived P and $\lambda$, archived instance 0 (251 targets, 4,367 calls, 2 spurious, 39 relations, log 2003), and the kernel being a kernel in reduced echelon form.
- **`tests/test_archived_identity.py`.** On **all 20 v3 points**, the binary's counts, log, ordered relations (`relations_raw`), relation digests, P and $\lambda$ equal run 39387473809's **Sage B rows** (PolyBoRi, the unmodified tau.sage). O2 equals run 38520305068's counts and logs and the v3 O1 relations and digests on all ten archived scalars.
- **`tests/test_pyrand.py`.** The native RNG equals CPython's `random` on every protocol seed and on 30 random and non-ASCII seeds. It covers seven `randrange` widths, 400 rounds of tauDlp's draw-and-shuffle stream, and shuffles of length 1 to 64. CI repeats this test in the Sage image's own Python 3.12.3.
- **`tests/test_n2x_compare.py`.** A pure-Python model of tau.sage (`tests/model_sage.py`) has its own independent echelon kernel. It is compared with the native trace item by item through the planted arm's own comparison code (`n2x_compare.py`): equal on the two smoke points and archived instance 0. Ten planted corruptions are each caught: a shift order, a relation sign, a kernel entry, a representative, a call count, a spurious count, the kernel index, a missing target, a missing base entry and $\lambda$.
- **`tests/test_verify_receipt.py`.** 20 of 20 pass on synthetic receipts. A clean receipt passes, and 19 planted faults fail closed.
- **`smoke/dryrun_model_sage.py`.** The real launcher, the real binary and the real verifier ran the whole smoke part. A stand-in `sage` answered with the model (`smoke/fake_sage.py`). Every smoke gate passed except `tier1_counters_recorded`, because this VM has no `/proc/schedstat`; GitHub runners have it (`smoke/dryrun_model_sage.json`).
- **Local cost split of N2** (cloud VM, a diagnostic, not evidence). A point took 74 ms. The v3 solve was about 15.5 $\mu$s per call, about 96% of that. The 2 × 306 scalar multiplications took about 3 ms and setup 0.15 ms. After this port, N2's cost is the unchanged v3 solve.

**The gates in the run.** The full set is in `protocol.json gates`.

- **S0.** v3's selftest runs on `libtau6loop.so`'s `tau6_*` solve. On 2,982 systems, Sage's variety equals the native one in order, equals the brute force, and equals a direct S3 evaluation.
- **S0b.** The binary's compiled solve returns Sage's variety, in Sage's order, on the same 2,982 systems.
- **O1 and O2.** O1 is the baseline on the ten archived scalars. O2 is N2 on them, compared with the archived counts and logs and the v3 O1 relations and digests.
- **Field check.** It runs in every native process. Sage arms run v3's `binding_check`, and N2X runs it on both libraries. Binary processes record the 256-product sample, and the verifier recomputes each product with v3's `pymul` modulo 854355.
- **Per point.** Targets, gb_calls, spurious, relations, log, ordered relations and digests are identical across B, N, N2, N2aa, N2X and the warm N2. The representatives are identical across the N2 processes and Sage's in N2X.
- **Planted arm N2X.** It runs Sage's path (the v3 N path) alongside N2's pipeline. It compares every target (al, be, R, shift order, calls, decomposition), the 29 representatives, all 33 base entries and $\lambda$. It also compares Sage's own `M.left_kernel().basis()` vector by vector, its index and the log, and aborts on any difference.
- **Smoke only.** On v3 points p00 and p01, every arm equals run 39387473809's archived Sage B rows.

## Primary: in-process cold CPU (and why)

- **Primary.** R = Σ cold CPU(N) / Σ cold CPU(N2) over the 120 primary pairs. Cold means setup + online, the process CPU clock inside each process: `time.process_time()` in Sage and `clock_gettime(CLOCK_PROCESS_CPUTIME_ID)` in tau6dlp, which is the same clock. The intervals are N's.
- **Why in-process.**
  1. It is the current row's measure: v3's primary and the ledger's cold `S`. The rho unit is in-process CPU too, so the ratchet stays in one unit.
  2. It is **conservative for this candidate**. It leaves out Sage start-up, which only N pays (about 1.2 s of child CPU per N process in v3, several times N's whole interval), so it can only understate N2's advantage.
  3. Start-up belongs to the interpreter, not to the algorithm.
- **Named process-inclusive secondary, pre-registered.** Child CPU N / N2 (user + system, `getrusage` of each launched process: start-up, intervals and post-interval work), over the same pairs with the same bootstrap. It gets its own verdict label but never decides. Launcher wall is reported beside it. This is the number a user who launches one solve would see. The CPU-in-process primary understates it.

## Frozen design

- **Panel.** 24 new scalars (`inputs/panel.json`, role `n19-native-loop-heldout`, seed `volcano-n19-native-loop-20261021:`
  None is equal to, the negative of, or a Frobenius image of an archived scalar, a v1, v2 or v3 panel scalar, or another panel scalar. The smoke points are v3 p00 and p01; they are never timed.
- **Order.** `schedule.json` holds 508 timed processes: S0, S0b, O1, O2, then per point (in a seeded shuffled block order):
  - 5 alternating N/N2 pairs with a seeded start;
  - 3 N2/N2aa pairs;
  - 1 N2X/N2 pair;
  - one B, one rho and one warm N2.

  The warm N2 is unpaired, so no paired process does post-interval batch work.
- **Interval.** A two-stage paired bootstrap (points, then pairs within each drawn point), 10,000 draws, `random.Random(2026102101)`, 95%. Improvement iff the lower bound is above 1, regression iff the upper bound is below 1.
- **Controls.**
  - A/A: N2/N2aa estimate inside [0.95, 1.05] over 72 pairs.
  - Planted: N2X/N2 lower bound above 1 over 24 pairs.
- **Secondaries.** These never decide:
  - the geometric mean of the pair ratios;
  - online, setup, cold wall, child CPU and launcher wall N/N2;
  - per-point ratios, start order and pair position;
  - `S` per arm and the N2 gap to rho;
  - the chain B / N2;
  - N2's phase shares;
  - warm means.

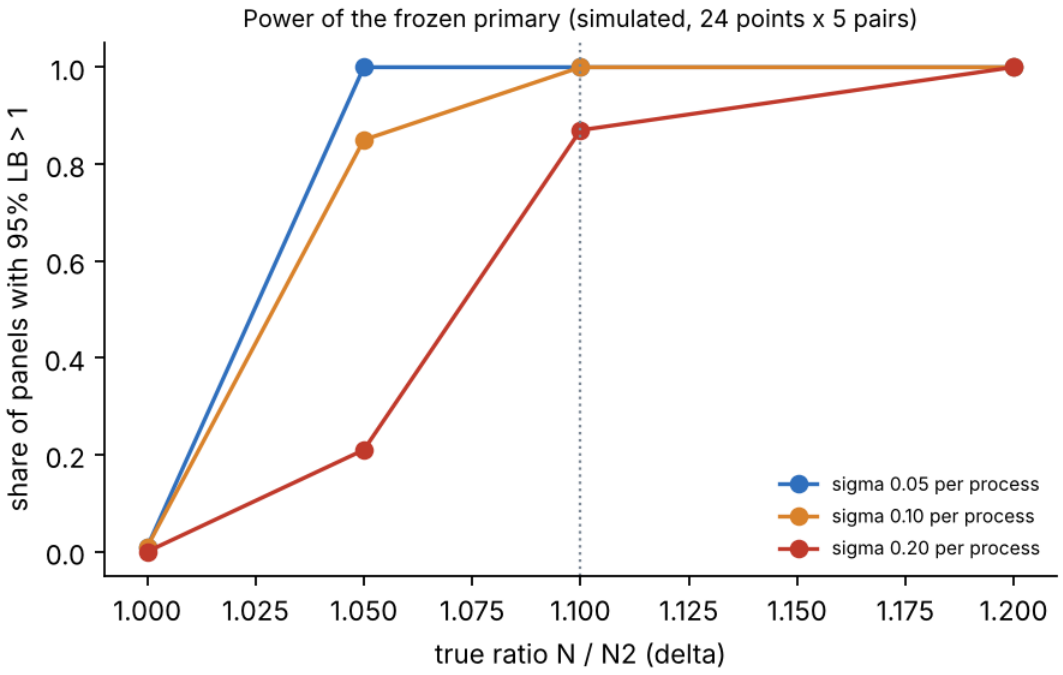



Figure 3: Power of the frozen primary (simulated). Vector: figures/power.svg.

- **Power** (`analysis/power.json`). The model uses per-point N costs from v3's N processes and log-normal per-process noise $\sigma = 0.05$, 0.10 or 0.20.
  - Power at a true ratio of 1.10 is at least 0.87 ($\sigma = 0.20$) and 1.00 for $\sigma \leq 0.10$.
  - The rejection rate with no effect is at most 0.01.
  - The A/A window fails in 0% of panels at $\sigma \leq 0.10$ and 17% at $\sigma = 0.20$. v3's A/A was 1.0003, which points to the low $\sigma$.
  - The expected effect is far larger: about 1 / 0.154 $\approx$ 6.5 if the port cost nothing.
- **Runtime and time guard.**
  - The smoke job takes about 15 min. The timed job takes about 35 min: the gates about 2 min, then per point about 65 s (B 35–65 s dominates).
  - The worst case, with B at 100 s, is about 50 min. Queueing included, the whole run should take about 1 to 1.5 h, inside the 3 h budget.
  - Before each segment, the guard skips it if the elapsed time plus max(its upper estimate, 1.1 $\times$ the longest point segment so far) exceeds 175 min. A skip makes the verdict invalid.
  - The job timeout is 200 min.
- **Tier.** Tier 1: one fresh GitHub-hosted ubuntu-24.04 runner, one pinned exposed CPU and one fresh process per run. The image is `sagemath/sagemath:10.10@sha256:310dfe23...`, and the three native artifacts are built with Rust 1.99.0. It is CPU-only, has one host class and is not replayed. A container affinity mask records placement, not host-wide exclusivity.

## Launch

- **Dispatch was tried first and refused.** `workflow_dispatch` through the GitHub integration returned 403 "Resource not accessible by integration"; the workflow is not on the default branch.
- **The push trigger launched it.** The push of the frozen commit to `claude/n19-native-loop` started the run, through the owner-approved path-limited trigger.
- **What does not trigger a run.** README, figures, `make_figures.py`, `report_header.tex`, `smoke/`, `diagnosis/` and `run_github_*/`.

## Next improvement

After this port the v3 solve is the cost: about 96% of N2 locally. It zero-fills a `[u32; 65536]` key buffer (256 KB) on every call while it uses at most 4,096 entries, and it decides all 4,096 (u, w) pairs by enumeration. Sizing the buffer to 2^(2k) and solving the linear system in w per u should cut it severalfold. S0 already pins down its output order.

## Files

- **Code.**
  - `native/`: the crate, with `src/bin/tau6dlp.rs` as arm N2;
  - `arm_n2x.sage` (the planted arm) and `n2x_compare.py`;
  - `run_schedule.py`, `verify_receipt.py` and `stats.py`.
- **Frozen inputs.**
  - `make_protocol.py` → `protocol.json`;
  - `make_panel.py` → `inputs/panel.json`;
  - `make_schedule.py` → `schedule.json`;
  - `power_check.py` → `analysis/power.json`.
- **Baseline, unchanged.** `../volcano-n19-native-tau6-v3-20261020/arm.sage` and its `native/`.
- **Checks.** `tests/` and `smoke/`.
- **Diagnosis.** `diagnosis/`.
- **Report.** `make_figures.py`, `figures/`, `README.pdf` and `report_header.tex`.